



Université Euromed de Fès

École d'Ingénierie Digitale et Intelligence Artificielle

Rapport du Projet Fin de Semestre

Système d'information d'aide à la décision et informatique
décisionnelle

Encadré par : Ahmed Ammamou

Préparé par : Marwa Hanini
Nada Ezzouhour El Achabi
Reda Jelfa
Mostapha Oulghazi

Table des matières

1	Introduction et Objectifs du Projet	3
1.1	Contexte général	3
1.2	Objectif du projet	3
1.3	Présentation du scénario métier : StreamPulse	3
1.4	Architecture globale de la solution	4
1.5	Indicateurs clés de performance (KPIs)	4
1.6	Valeur ajoutée du projet	4
2	Architecture Globale du Pipeline	5
2.1	Vue d'ensemble de l'architecture	5
2.2	Rôle de chaque composant	5
2.2.1	PostgreSQL – Système transactionnel (OLTP)	5
2.2.2	Amazon S3 – Data Lake (zone RAW)	5
2.2.3	Snowflake – Data Warehouse Cloud	5
2.2.4	Power BI – Couche de visualisation	6
2.3	Avantages de l'architecture	6
3	Génération des Données dans PostgreSQL	7
3.1	Contexte et justification	7
3.2	Objectifs de la génération des données	7
3.3	Choix de PostgreSQL comme base OLTP	7
3.3.1	Structure et paramètres du script	7
3.3.2	Script de génération des données	8
3.4	Modélisation des entités métier	9
3.4.1	Table users	9
3.4.2	Table contents	9
3.4.3	Table episodes	10
3.4.4	Table subscriptions	10
3.4.5	Table viewing_events	11
3.5	Lien avec les étapes suivantes	11
4	Export des Données vers Amazon S3 (Data Lake)	13
4.1	Rôle d'Amazon S3 dans le pipeline	13
4.2	Organisation et structure du Data Lake	13
4.3	Choix du format CSV	14
4.4	Mécanisme d'export depuis PostgreSQL	14
4.5	Lien avec les étapes suivantes	15
5	Configuration de Snowflake (Data Warehouse)	16
5.1	Rôle de Snowflake dans l'architecture StreamPulse	16
5.2	Création de la base de données analytique	16
5.3	Organisation logique en schémas	17
5.4	Intégration sécurisée avec Amazon S3	17
5.5	Création du stage externe Amazon S3	18
5.6	Chargement des données dans le schéma RAW	18
5.7	Nettoyage et enrichissement des données (STAGING)	18

5.8	Création des Data Marts analytiques	19
6	Orchestration du pipeline avec Apache Airflow	21
6.1	Rôle d'Apache Airflow dans l'architecture StreamPulse	21
6.2	Choix d'un déploiement Airflow avec Docker	21
6.3	Installation et initialisation d'Airflow	21
6.4	Configuration Docker de l'environnement Airflow	22
6.5	Gestion des dépendances Airflow	22
6.6	Définition du DAG StreamPulse	23
6.6.1	Structure générale du DAG	23
6.7	Description des tâches du pipeline	23
6.7.1	Extraction depuis PostgreSQL	23
6.7.2	Chargement vers Amazon S3	23
6.7.3	Chargement dans Snowflake	23
6.7.4	Transformation avec dbt	24
6.8	Chaînage des tâches	24
6.9	Supervision et suivi des exécutions	25
7	Data Mart et Visualisation des Données	26
7.1	Objectif de la phase Data Mart et BI	26
7.2	Construction du Data Mart dans Snowflake	26
7.2.1	Organisation des schémas	26
7.2.2	Table de faits	26
7.2.3	Tables de dimensions	26
7.2.4	Architecture en étoile	27
7.3	Connexion de Power BI à Snowflake	27
7.4	Modélisation des données dans Power BI	27
7.5	Visualisation et Indicateurs Clés	27
7.5.1	Indicateurs clés (KPIs)	27
7.5.2	Analyses graphiques	27

1 Introduction et Objectifs du Projet

1.1 Contexte général

Avec l'essor des plateformes numériques, les entreprises sont aujourd'hui confrontées à des volumes de données massifs, hétérogènes et générés en continu. Les plateformes de streaming vidéo, telles que Netflix, YouTube ou Amazon Prime Video, produisent quotidiennement des millions d'événements liés au comportement des utilisateurs : lectures, pauses, abandons, complétions, changements de périphériques, etc.

Dans ce contexte, la capacité à collecter, stocker, transformer et analyser efficacement ces données devient un levier stratégique majeur. Les entreprises doivent être capables de transformer des données brutes issues de systèmes transactionnels en indicateurs décisionnels fiables, exploitables par les équipes métier et les décideurs.

C'est dans cette optique que s'inscrit le projet **StreamPulse**, dont l'objectif est de mettre en œuvre une *Modern Data Stack* complète, automatisée et orientée analyse.

1.2 Objectif du projet

L'objectif principal de ce projet est de concevoir et implémenter une chaîne complète de traitement des données permettant :

- d'ingérer des données brutes issues d'un système transactionnel (PostgreSQL),
- de stocker ces données dans un Data Lake basé sur Amazon S3,
- de charger et structurer les données dans un Data Warehouse Cloud (Snowflake),
- d'automatiser les flux de données à l'aide d'Apache Airflow,
- de produire des indicateurs analytiques prêts à être consommés par un outil de Business Intelligence (Power BI).

Le projet vise ainsi à reproduire une architecture professionnelle proche de celles utilisées dans les entreprises data-driven modernes.

1.3 Présentation du scénario métier : StreamPulse

StreamPulse est une plateforme SaaS de streaming vidéo fictive, inspirée des plateformes de streaming actuelles. Elle propose à ses utilisateurs des films, séries et épisodes accessibles via différents types de périphériques (mobile, web, smart TV, tablette, etc.).

La plateforme génère plusieurs types de données :

- des données utilisateurs (profils, abonnements),
- des données de catalogue (contenus, épisodes),
- des données d'événements de visionnage (logs de lecture).

Ces données sont initialement stockées dans une base PostgreSQL jouant le rôle de système transactionnel (*OLTP*).

1.4 Architecture globale de la solution

L'architecture mise en place dans le cadre du projet repose sur les composants suivants :

PostgreSQL (OLTP) → Amazon S3 (Data Lake) → Snowflake (DWH) → Power BI (BI)

L'orchestration de l'ensemble du pipeline est assurée par **Apache Airflow**, qui automatise l'exécution des différentes étapes du flux de données.

- **PostgreSQL** : source des données transactionnelles.
- **Amazon S3** : zone de stockage des données brutes (RAW).
- **Snowflake** : Data Warehouse pour l'analyse et l'agrégation des données.
- **Apache Airflow** : orchestration et automatisation du pipeline.
- **Power BI** : visualisation et analyse décisionnelle.

Cette architecture correspond aux standards actuels de la *Modern Data Stack*.

1.5 Indicateurs clés de performance (KPIs)

À partir des données de visionnage et des abonnements, plusieurs indicateurs stratégiques sont calculés afin d'évaluer la performance de la plateforme StreamPulse :

- le taux de complétion des vidéos,
- la répartition des sessions de visionnage par type de périphérique,
- les contenus les plus visionnés,
- la durée totale de visionnage,
- le revenu mensuel récurrent (*Monthly Recurring Revenue – MRR*).

Ces indicateurs sont matérialisés sous forme de vues analytiques dans Snowflake et visualisés à l'aide de tableaux de bord interactifs Power BI.

—

1.6 Valeur ajoutée du projet

Ce projet permet de démontrer concrètement :

- la mise en œuvre d'une architecture Big Data moderne,
- la séparation claire entre données brutes, données transformées et données analytiques,
- l'importance de l'orchestration des pipelines de données,
- la transformation de données techniques en informations décisionnelles exploitables.

Il constitue ainsi une application pratique des concepts de Data Engineering, Data Warehousing et Business Intelligence étudiés durant le cursus.

2 Architecture Globale du Pipeline

2.1 Vue d'ensemble de l'architecture

L'architecture mise en place dans le cadre du projet **StreamPulse** repose sur un pipeline de données moderne, conçu pour assurer la collecte, le stockage, la transformation et la visualisation des données de manière fiable et scalable.

Le pipeline suit une approche séquentielle, dans laquelle chaque composant joue un rôle précis dans la chaîne de valeur de la donnée, depuis sa génération jusqu'à sa consommation analytique.

PostgreSQL (OLTP) → Export CSV → Amazon S3 (Data Lake - RAW) → Snowflake (RAW → STAGING → MARTS) → Power BI (Dashboard Analytics)

Cette architecture correspond aux standards actuels des plateformes analytiques utilisées en entreprise.

2.2 Rôle de chaque composant

2.2.1 PostgreSQL – Système transactionnel (OLTP)

PostgreSQL joue le rôle de base de données transactionnelle (*Online Transaction Processing*). Il simule le système d'information principal de la plateforme StreamPulse, où sont stockées les données opérationnelles :

- utilisateurs,
- abonnements,
- contenus et épisodes,
- événements de visionnage.

Ces données sont optimisées pour l'écriture et les transactions rapides, mais ne sont pas adaptées à l'analyse analytique à grande échelle.

2.2.2 Amazon S3 – Data Lake (zone RAW)

Amazon S3 est utilisé comme *Data Lake*, servant de zone de stockage intermédiaire pour les données brutes. Les données extraites de PostgreSQL sont exportées au format CSV et stockées sans transformation dans S3.

Cette approche permet :

- de conserver une copie brute et historisée des données,
- de découpler la source transactionnelle du Data Warehouse,
- de faciliter la reprise ou le rechargement des données.

2.2.3 Snowflake – Data Warehouse Cloud

Snowflake constitue le cœur analytique du pipeline. Il est organisé en plusieurs couches logiques :

- **RAW** : données chargées telles quelles depuis S3,
- **STAGING** : données nettoyées et normalisées,
- **MARTS** : données agrégées et optimisées pour la BI.

Cette structuration garantit la lisibilité, la maintenabilité et la performance des requêtes analytiques.

2.2.4 Power BI – Couche de visualisation

Power BI est utilisé pour exploiter les données finales présentes dans les *Data Marts*. Il permet de créer des tableaux de bord interactifs destinés à l'analyse métier et à l'aide à la décision.

Les indicateurs produits sont directement connectés aux vues analytiques de Snowflake.

2.3 Avantages de l'architecture

Cette architecture présente plusieurs avantages :

- séparation claire des responsabilités entre les composants,
- forte scalabilité,
- facilité d'évolution et de maintenance,
- conformité avec les bonnes pratiques de la Modern Data Stack.

3 Génération des Données dans PostgreSQL

3.1 Contexte et justification

Avant de mettre en place un pipeline analytique complet, il est indispensable de disposer d'un jeu de données suffisamment riche et réaliste pour simuler le fonctionnement réel d'une plateforme de streaming.

Dans un contexte professionnel, ces données proviendraient de systèmes en production. Dans le cadre de ce projet académique, les données ont été générées artificiellement afin de reproduire des comportements utilisateurs plausibles et de tester l'ensemble de la chaîne de traitement de bout en bout.

PostgreSQL a été choisi comme système transactionnel (*OLTP*) afin de simuler la base de données opérationnelle de la plateforme StreamPulse.

3.2 Objectifs de la génération des données

La génération des données poursuit plusieurs objectifs :

- simuler l'activité d'une plateforme de streaming réelle,
- produire un volume de données suffisant pour des analyses statistiques pertinentes,
- tester la robustesse du pipeline d'ingestion et de transformation,
- fournir une base fiable pour le calcul des indicateurs métiers.

Les données générées doivent être cohérentes entre elles et respecter les relations métier entre les différentes entités (utilisateurs, contenus, abonnements, événements).

3.3 Choix de PostgreSQL comme base OLTP

PostgreSQL est utilisé comme base de données transactionnelle pour les raisons suivantes :

- forte fiabilité et stabilité,
- conformité aux standards SQL,
- gestion efficace des transactions,
- large adoption dans les environnements professionnels.

Dans l'architecture StreamPulse, PostgreSQL représente le système source à partir duquel les données seront extraites vers le Data Lake.

3.3.1 Structure et paramètres du script

Le script `generate_streaming_data.py` permet de contrôler facilement le volume de données à générer à l'aide de constantes configurables :

- 500 utilisateurs

- 100 contenus
- 300 épisodes
- 200 abonnements
- 10 000 événements de visionnage

Les données sont directement insérées dans le schéma raw de la base PostgreSQL.

3.3.2 Script de génération des données

Listing 3.1 – Connexion à la base PostgreSQL

```
DB_CONFIG = {
    'host': 'localhost',
    'port': 5432,
    'database': 'StreamPulse',
    'user': 'postgres',
    'password': '*****'
}

def get_connection():
    return psycopg2.connect(**DB_CONFIG)
```

Listing 3.2 – Génération des utilisateurs

```
def generate_users(conn):
    cur = conn.cursor()
    users = []
    for _ in range(NB_USERS):
        users.append((
            fake.user_name(),
            fake.email(),
            random.choice(COUNTRIES),
            random.choice(DEVICES)
        ))
    execute_batch(
        cur,
        "INSERT INTO raw.users (username,email,country,device_type) VALUES (%s,%s,%s,%s)",
        users
    )
    conn.commit()
```

Listing 3.3 – Génération des événements de visionnage

```
def generate_viewing_events(conn):
    cur = conn.cursor()

    cur.execute("SELECT user_id FROM raw.users")
    users = [u[0] for u in cur.fetchall()]

    cur.execute("SELECT content_id FROM raw.contents")
    contents = [c[0] for c in cur.fetchall()]

    events = []
    for _ in range(NB_VIEWING_EVENTS):
        start_time = fake.date_time_between(start_date='-6m', end_date='now')
        watched = random.randint(60, 7200)
        completion = round(min(watched / random.randint(1800,10800), 1), 2)

        events.append((
            random.choice(users),
            random.choice(contents),
            start_time,
            completion
        ))

    execute_batch(
        cur,
        "INSERT INTO raw.viewing_events (user_id,content_id,start_time,completion) VALUES (%s,%s,%s,%s)",
        events
    )
    conn.commit()
```

```

        watched,
        completion,
        random.choice(DEVICES)
    ))

execute_batch(
    cur,
    """
    INSERT INTO raw.viewing_events
    (user_id, content_id, start_timestamp, watched_duration_seconds,
     completion_rate, device)
    VALUES (%s,%s,%s,%s,%s,%s)
    """ ,
    events,
    page_size=500
)
conn.commit()

```

3.4 Modélisation des entités métier

Les données générées sont structurées autour des principales entités métier d'une plateforme de streaming.

3.4.1 Table users

Cette table contient les informations relatives aux utilisateurs de la plateforme :

- identifiant utilisateur,
- informations de profil,
- pays,
- date de création du compte.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
1	aldoreen	aldoreen@example.com	2026-01-04 15:21:13	769723	Japan	chromecast															
2	zflynn	zflynn@example.com	2026-01-04 15:21:13	769723	France	chromecast															
3	thyasota	thyasota@example.com	2026-01-04 15:21:13	769723	Canada	smart_tv															
4	zwing	zwing@example.com	2026-01-04 15:21:13	769723	United States	chromecast															
5	levans	levans@example.com	2026-01-04 15:21:13	769723	Germany	web															
6	larrybarrett	larrybarrett@example.com	2026-01-04 15:21:13	769723	Spain	mobile															
7	zaweney	zaweney@example.com	2026-01-04 15:21:13	769723	Brazil	tablet															
8	nath83	nath83@example.com	2026-01-04 15:21:13	769723	Canada	smart_tv															
9	pconesa	pconesa@example.com	2026-01-04 15:21:13	769723	Canada	mobile															
10	diana21	diana21@example.com	2026-01-04 15:21:13	769723	United States	chromecast															
11	lwehner	lwehner@example.com	2026-01-04 15:21:13	769723	France	chromecast															
12	charlie28	charlie28@example.com	2026-01-04 15:21:13	769723	United Kingdom	smart_tv															
13	qcharrier	qcharrier@example.com	2026-01-04 15:21:13	769723	Japan	smart_tv															
14	monica	monica@example.com	2026-01-04 15:21:13	769723	France	mobile															
15	satoshi	satoshi@example.com	2026-01-04 15:21:13	769723	Canada	chromecast															
16	kimberly15	kimberly15@example.com	2026-01-04 15:21:13	769723	Germany	mobile															
17	morgaghirenzo	morgaghirenzo@example.com	2026-01-04 15:21:13	769723	Spain	tablet															
18	msuzuki	msuzuki@example.com	2026-01-04 15:21:13	769723	France	web															
19	ghyys	ghyys@example.com	2026-01-04 15:21:13	769723	Spain	mobile															
20	garryclarke	garryclarke@example.com	2026-01-04 15:21:13	769723	France	mobile															
21	lewisabbie	lewisabbie@example.com	2026-01-04 15:21:13	769723	United Kingdom	smart_tv															
22	rverdier	rverdier@example.com	2026-01-04 15:21:13	769723	France	chromecast															
23	tenenillo	tenenillo@example.com	2026-01-04 15:21:13	769723	Spain	mobile															
24	mturur	mturur@example.com	2026-01-04 15:21:13	769723	France	mobile															
25	gabrieleandrea	gabrieleandrea@example.com	2026-01-04 15:21:13	769723	United Kingdom	chromecast															
26	ipatterson	ipatterson@example.com	2026-01-04 15:21:13	769723	Japan	mobile															
27	grillmann	grillmann@example.com	2026-01-04 15:21:13	769723	Brazil	mobile															
28	craigchristina	craigchristina@example.com	2026-01-04 15:21:13	769723	United Kingdom	mobile															
29	davidgosselin	davidgosselin@example.com	2026-01-04 15:21:13	769723	Brazil	tablet															
30	margaret17	margaret17@example.com	2026-01-04 15:21:13	769723	Canada	mobile															
31	shohin32	shohin32@example.com	2026-01-04 15:21:13	769723	Germany	tablet															
32	jasi	jasi@example.com	2026-01-04 15:21:13	769723	Canada	mobile															
33	pruschiehanny	pruschiehanny@example.com	2026-01-04 15:21:13	769723	Spain	mobile															
34	donna81	donna81@example.com	2026-01-04 15:21:13	769723	Brazil	chromecast															
35	brunmillier	brunmillier@example.com	2026-01-04 15:21:13	769723	United Kingdom	chromecast															
36	thun28	thun28@example.com	2026-01-04 15:21:13	769723	Germany	tablet															

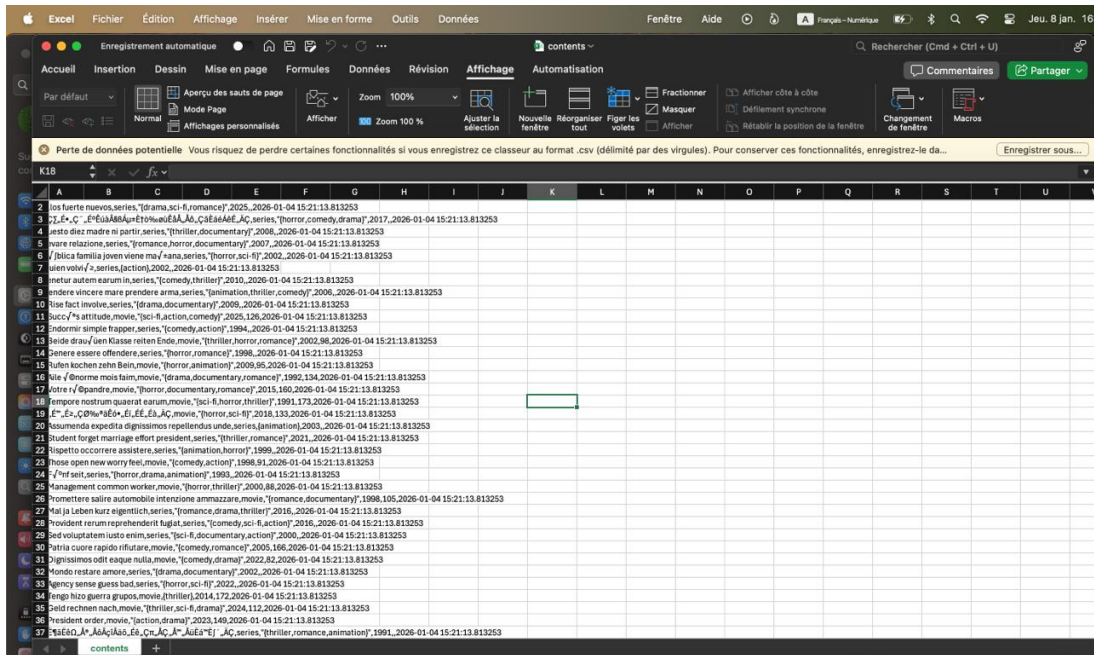
3.4.2 Table contents

La table contents représente le catalogue de la plateforme. Elle contient les films et les séries proposés aux utilisateurs.

Chaque contenu est caractérisé par :

- un identifiant unique,

- un type (film ou série),
- une catégorie,
- une durée ou un nombre d'épisodes.



3.4.3 Table episodes

Pour les contenus de type série, la table `episodes` stocke les épisodes associés.

Cette séparation permet de modéliser correctement les relations entre séries et épisodes, tout en conservant une structure relationnelle claire.

[illegible]

3.4.4 Table subscriptions

La table subscriptions contient les informations liées aux abonnements des utilisateurs :

- type de plan (Basic, Standard, Premium),
- date de début et de fin.

— statut de l'abonnement.

[illegible]

3.4.5 Table viewing_events

La table `viewing_events` constitue la table la plus volumineuse et la plus stratégique du projet.

Elle enregistre chaque session de visionnage effectuée par un utilisateur et contient notamment :

- l'utilisateur,
- le contenu ou l'épisode visionné,
- l'horodatage de début,
- la durée de visionnage,
- le taux de complétion,
- le type de périphérique utilisé.

</

3.5 Lien avec les étapes suivantes

Cette étape de génération des données constitue le point de départ du pipeline StreamPulse. Les données produites dans PostgreSQL seront ensuite :

- exportées vers Amazon S3 sous forme de fichiers CSV,
- chargées dans Snowflake pour analyse,
- transformées en indicateurs métiers,
- visualisées à travers des tableaux de bord Power BI.

4 Export des Données vers Amazon S3 (Data Lake)

4.1 Rôle d'Amazon S3 dans le pipeline

Dans l'architecture du projet **StreamPulse**, Amazon S3 joue le rôle de *Data Lake*, c'est-à-dire une zone de stockage intermédiaire destinée à conserver les données brutes issues du système transactionnel.

Contrairement au Data Warehouse, le Data Lake ne vise pas à optimiser les performances analytiques, mais à stocker les données telles qu'elles sont produites, sans transformation préalable. Cette approche est conforme aux principes de la *Modern Data Stack*.

L'utilisation d'Amazon S3 comme zone **RAW** présente plusieurs avantages :

- **Scalabilité** : capacité à stocker de grands volumes de données sans contrainte de taille,
- **Découplage stockage / calcul** : les données sont stockées indépendamment du moteur analytique,
- **Interopérabilité** : intégration native avec Snowflake, Airflow et les outils Big Data,
- **Traçabilité** : conservation d'une version brute des données pour audit ou rechargement.

Amazon S3 constitue ainsi une étape clé entre la base transactionnelle PostgreSQL et le Data Warehouse Snowflake.

4.2 Organisation et structure du Data Lake

Afin d'assurer une bonne lisibilité et une évolutivité du Data Lake, une structure hiérarchique claire est mise en place dans le bucket S3.

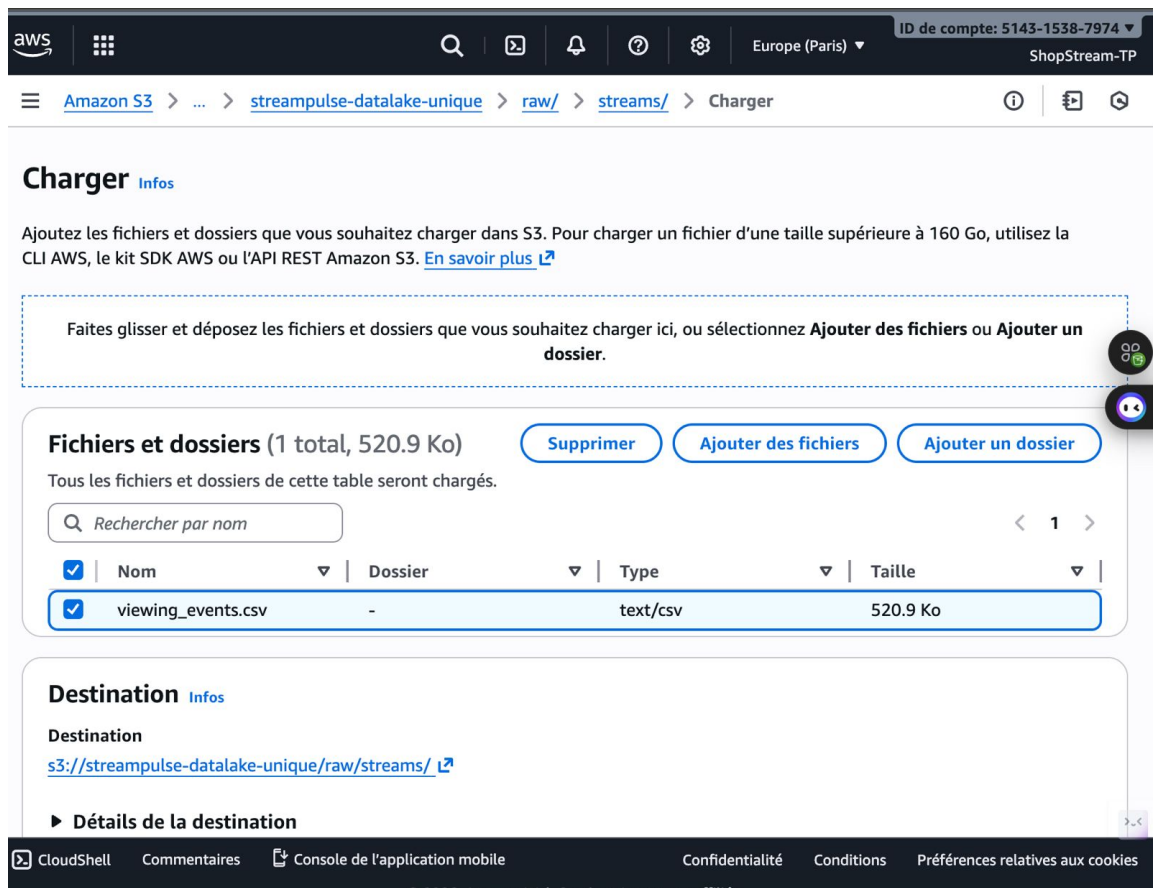
La structure retenue est la suivante :

```
s3://streampulse-datalake-unique/  
    raw/  
        streams/  
            viewing_events.csv
```

Cette organisation respecte les bonnes pratiques suivantes :

- séparation explicite de la zone `raw`,
- regroupement des données par domaine métier (`streams`),
- stockage des données au format fichier, indépendamment du schéma cible.

Cette structure pourra être enrichie ultérieurement avec d'autres sous-dossiers (`users`, `subscriptions`, `contents`, etc.).



4.3 Choix du format CSV

Les données sont exportées vers Amazon S3 au format **CSV** (Comma-Separated Values).

Ce choix est justifié par plusieurs raisons :

- format simple et lisible,
- compatibilité native avec Snowflake,
- facilité de génération depuis PostgreSQL,
- prise en charge par Amazon S3 Select et autres outils analytiques.

Chaque fichier CSV contient une ligne d'en-tête décrivant les colonnes, facilitant ainsi le chargement ultérieur dans Snowflake.

4.4 Mécanisme d'export depuis PostgreSQL

Les données stockées dans PostgreSQL sont extraites à l'aide d'un script Python dédié. Ce script se charge de :

- se connecter à la base PostgreSQL,
- lire les données de la table `viewing_events`,
- convertir les résultats en fichier CSV,
- déposer le fichier dans le bucket Amazon S3.

L'export est réalisé sans transformation afin de préserver l'intégrité des données sources.

☐	PID	Type	Server	Object	Start Time ▾	Status	Time Taken (sec)
☐	☒ 35140	Export Data	PostgreSQL 15 (loca...	StreamPulse/raw.co...	06/01/2026 16:50:14	Finished	0.08
☐	☒ 34996	Export Data	PostgreSQL 15 (loca...	StreamPulse/raw.epi...	06/01/2026 16:49:58	Finished	0.08
☐	☒ 16324	Export Data	PostgreSQL 15 (loca...	StreamPulse/raw.su...	06/01/2026 16:49:40	Finished	0.08
☐	☒ 8104	Export Data	PostgreSQL 15 (loca...	StreamPulse/raw.us...	06/01/2026 16:49:19	Finished	0.09
☐	☒ 21700	Export Data	PostgreSQL 15 (loca...	StreamPulse/raw.vie...	04/01/2026 16:47:46	Finished	0.23

4.5 Lien avec les étapes suivantes

L'export des données vers Amazon S3 marque la fin de la phase d'ingestion brute du pipeline.

À partir de cette étape :

- Snowflake pourra accéder directement aux fichiers S3 via une *Storage Integration*,
- les données seront chargées dans le schéma RAW du Data Warehouse,
- des transformations seront appliquées dans les couches STAGING et MARTS.

Amazon S3 joue ainsi un rôle central dans la transition entre données transactionnelles et données analytiques.

5 Configuration de Snowflake (Data Warehouse)

5.1 Rôle de Snowflake dans l'architecture StreamPulse

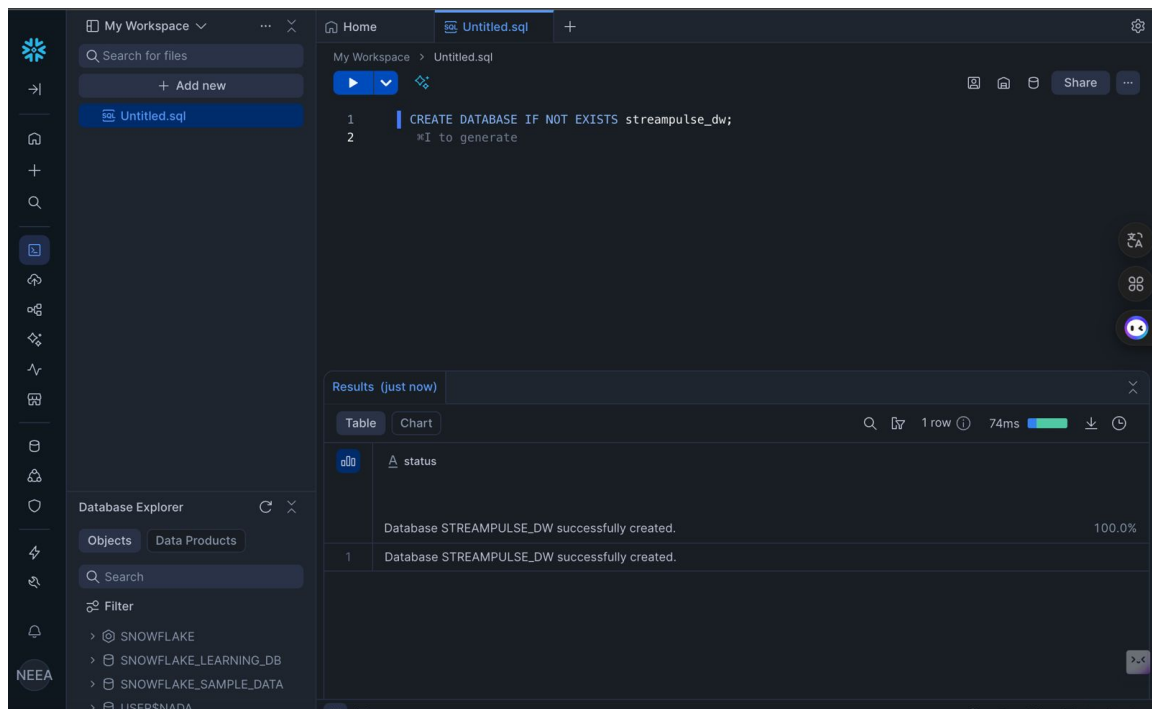
Snowflake est utilisé dans le projet **StreamPulse** comme *Data Warehouse Cloud*. Il constitue la couche analytique centrale du pipeline de données, assurant le stockage, la structuration et l'analyse des données issues du Data Lake Amazon S3.

Contrairement à une base de données transactionnelle classique, Snowflake est optimisé pour :

- le traitement analytique de grands volumes de données,
- l'exécution de requêtes OLAP complexes,
- la séparation entre stockage et calcul,
- l'accès concurrent par plusieurs outils de Business Intelligence.

Snowflake permet ainsi de transformer les données brutes de StreamPulse en indicateurs décisionnels fiables et exploitables.

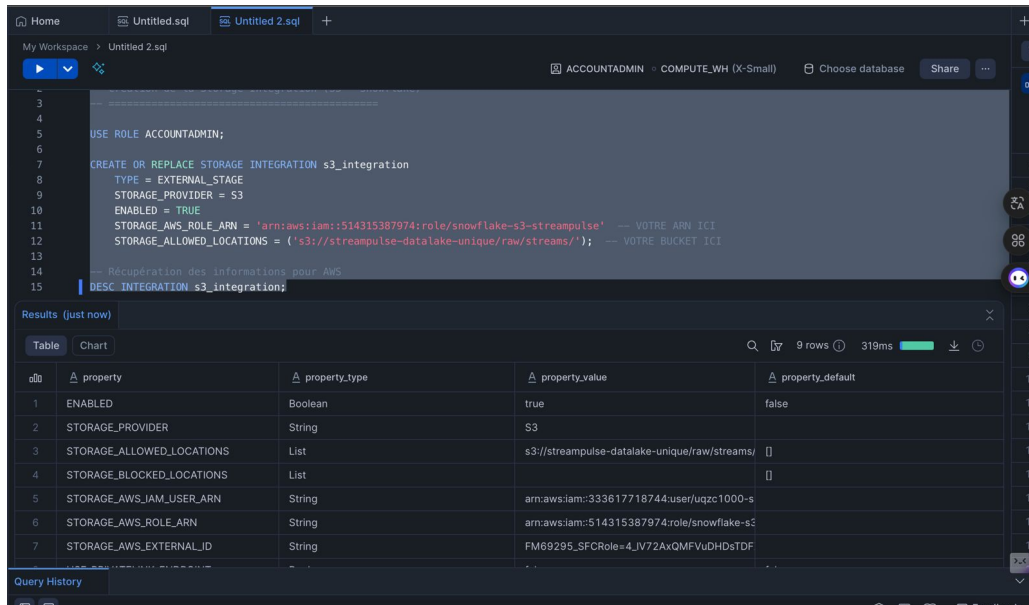
5.2 Création de la base de données analytique



5.3 Organisation logique en schémas

Afin de respecter les bonnes pratiques de modélisation analytique, la base de données est organisée en plusieurs schémas logiques correspondant aux différentes couches du pipeline.

- **RAW** : données brutes chargées depuis Amazon S3,
- **STAGING** : données nettoyées et enrichies,
- **MARTS** : vues analytiques finales destinées à la BI.

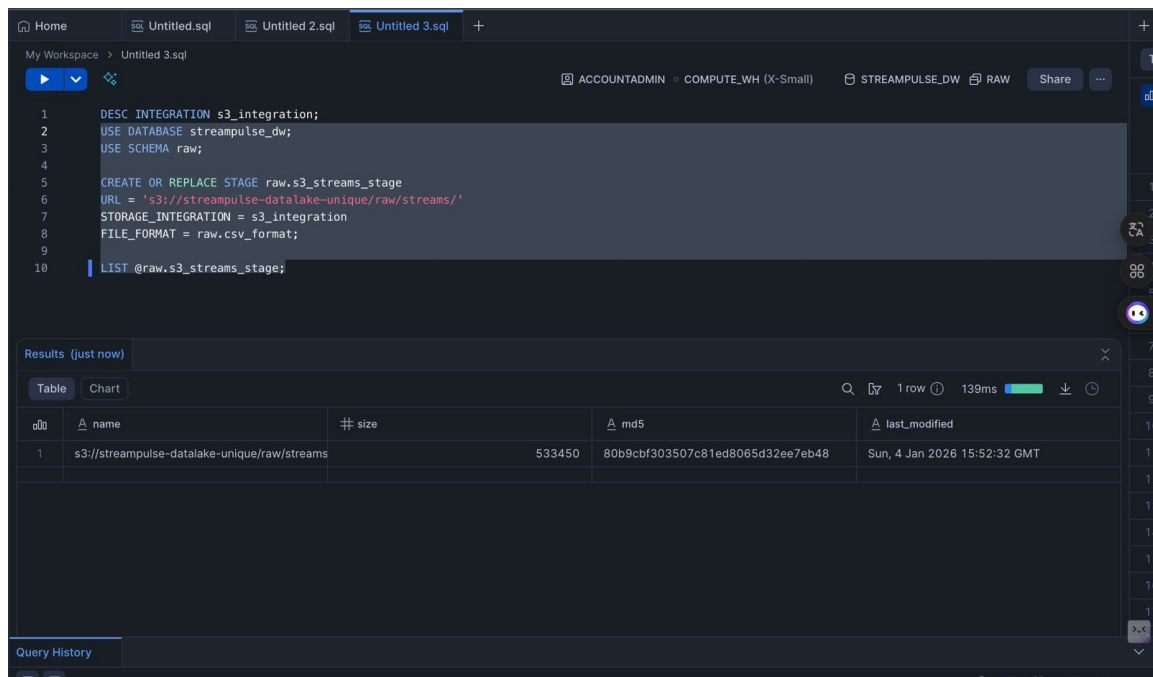


```
1  -----
2
3  USE ROLE ACCOUNTADMIN;
4
5
6
7  CREATE OR REPLACE STORAGE INTEGRATION s3_integration
8  TYPE = EXTERNAL_STAGE
9  STORAGE_PROVIDER = S3
10  ENABLED = TRUE
11  STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::514315387974:role/snowflake-s3-streampulse' -- VOTRE ARN ICI
12  STORAGE_ALLOWED_LOCATIONS = ('s3://streampulse-datalake-unique/raw/streams/'); -- VOTRE BUCKET ICI
13
14  -- Récupération des informations pour AWS
15  DESC INTEGRATION s3_integration;
```

Results (just now)

property	property_type	property_value	property_default
ENABLED	Boolean	true	false
STORAGE_PROVIDER	String	S3	
STORAGE_ALLOWED_LOCATIONS	List	s3://streampulse-datalake-unique/raw/streams/	[]
STORAGE_BLOCKED_LOCATIONS	List		[]
STORAGE_AWS_IAM_USER_ARN	String	arn:aws:iam::333617718744:user/uqzc1000-s	
STORAGE_AWS_ROLE_ARN	String	arn:aws:iam::514315387974:role/snowflake-s3	
STORAGE_AWS_EXTERNAL_ID	String	FM69295_SFCRole=4_JV72AxQMFVuDHDsTDF	

5.4 Intégration sécurisée avec Amazon S3

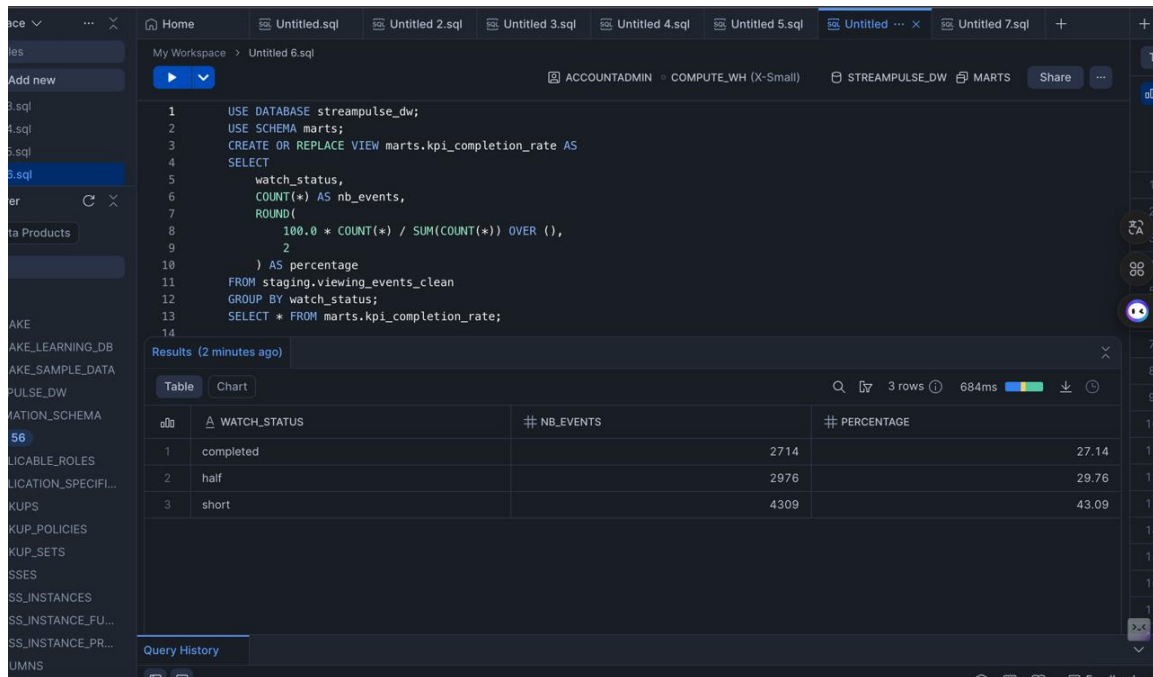


```
1  DESC INTEGRATION s3_integration;
2  USE DATABASE streampulse_dw;
3  USE SCHEMA raw;
4
5  CREATE OR REPLACE STAGE raw.s3_streams_stage
6  URL = 's3://streampulse-datalake-unique/raw/streams/'
7  STORAGE_INTEGRATION = s3_integration
8  FILE_FORMAT = raw.csv_format;
9
10 LIST @raw.s3_streams_stage;
```

Results (just now)

name	size	md5	last_modified
s3://streampulse-datalake-unique/raw/streams	533450	80b9cbf303507c81ed8065d32ee7eb48	Sun, 4 Jan 2026 15:52:32 GMT

5.5 Création du stage externe Amazon S3



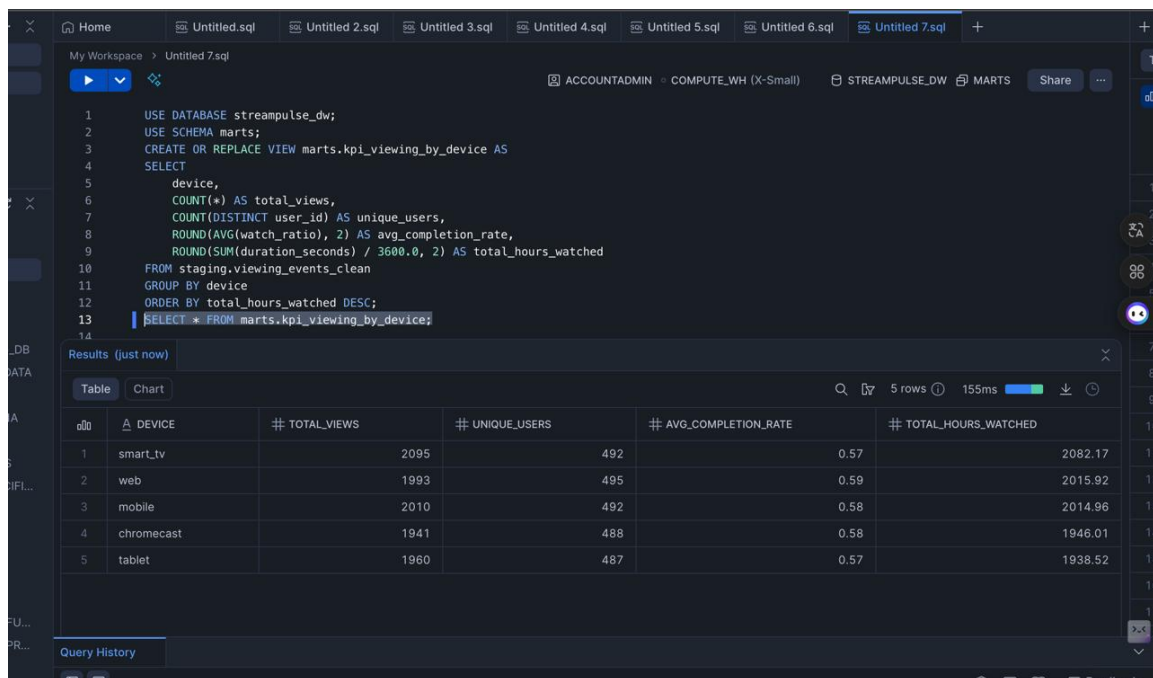
The screenshot shows a SQL IDE interface with a query editor and a results pane. The query editor contains the following SQL code:

```
1 USE DATABASE streampulse_dw;
2 USE SCHEMA marts;
3 CREATE OR REPLACE VIEW marts.kpi_completion_rate AS
4 SELECT
5     watch_status,
6     COUNT(*) AS nb_events,
7     ROUND(
8         100.0 * COUNT(*) / SUM(COUNT(*) OVER ()),
9         2
10    ) AS percentage
11 FROM staging.viewing_events_clean
12 GROUP BY watch_status;
13 SELECT * FROM marts.kpi_completion_rate;
```

The results pane shows the output of the query, which is a table with 3 rows and 4 columns: WATCH_STATUS, NB_EVENTS, and PERCENTAGE. The data is as follows:

WATCH_STATUS	NB_EVENTS	PERCENTAGE
completed	2714	27.14
half	2976	29.76
short	4309	43.09

5.6 Chargement des données dans le schéma RAW



The screenshot shows a SQL IDE interface with a query editor and a results pane. The query editor contains the following SQL code:

```
1 USE DATABASE streampulse_dw;
2 USE SCHEMA marts;
3 CREATE OR REPLACE VIEW marts.kpi_viewing_by_device AS
4 SELECT
5     device,
6     COUNT(*) AS total_views,
7     COUNT(DISTINCT user_id) AS unique_users,
8     ROUND(AVG(watch_ratio), 2) AS avg_completion_rate,
9     ROUND(SUM(duration_seconds) / 3600.0, 2) AS total_hours_watched
10 FROM staging.viewing_events_clean
11 GROUP BY device
12 ORDER BY total_hours_watched DESC;
13 SELECT * FROM marts.kpi_viewing_by_device;
```

The results pane shows the output of the query, which is a table with 5 rows and 5 columns: DEVICE, TOTAL_VIEWS, UNIQUE_USERS, AVG_COMPLETION_RATE, and TOTAL_HOURS_WATCHED. The data is as follows:

DEVICE	TOTAL_VIEWS	UNIQUE_USERS	AVG_COMPLETION_RATE	TOTAL_HOURS_WATCHED
smart_tv	2095	492	0.57	2082.17
web	1993	495	0.59	2015.92
mobile	2010	492	0.58	2014.96
chromecast	1941	488	0.58	1946.01
tablet	1960	487	0.57	1938.52

5.7 Nettoyage et enrichissement des données (STAGING)

Une vue de nettoyage a été créée afin de :

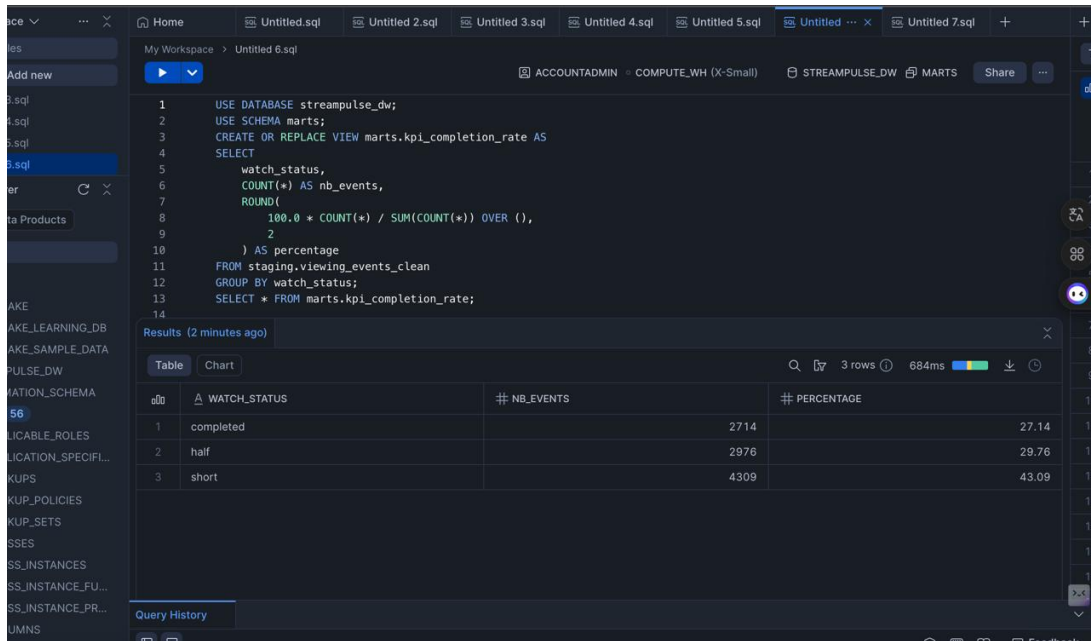
- corriger les valeurs manquantes,
- recalculer les dates de fin de visionnage,
- classer le comportement utilisateur.

5.8 Création des Data Marts analytiques

Les *Data Marts* représentent la couche finale destinée à l'analyse décisionnelle.

Trois vues analytiques principales ont été créées :

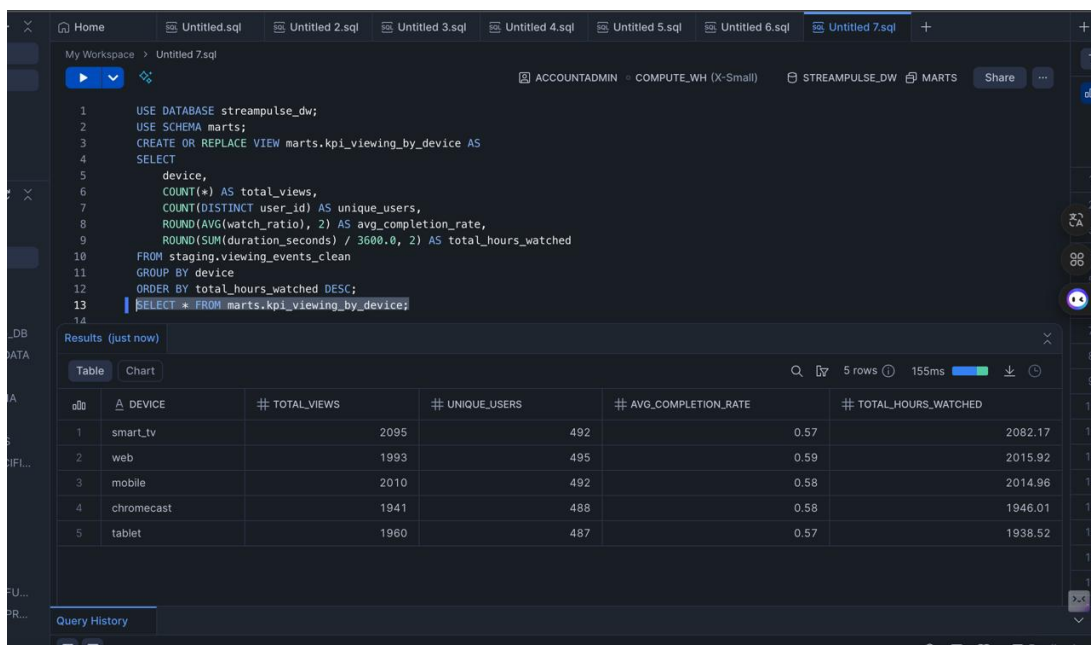
- le taux de complétion des vidéos,
- l'analyse par type de périphérique,
- le top des contenus les plus visionnés.



```
1 USE DATABASE streampulse_dw;
2 USE SCHEMA marts;
3 CREATE OR REPLACE VIEW marts.kpi_completion_rate AS
4 SELECT
5     watch_status,
6     COUNT(*) AS nb_events,
7     ROUND(
8         100.0 * COUNT(*) / SUM(COUNT(*) OVER ()),
9         2
10    ) AS percentage
11 FROM staging.viewing_events_clean
12 GROUP BY watch_status;
13 SELECT * FROM marts.kpi_completion_rate;
```

Results (2 minutes ago)

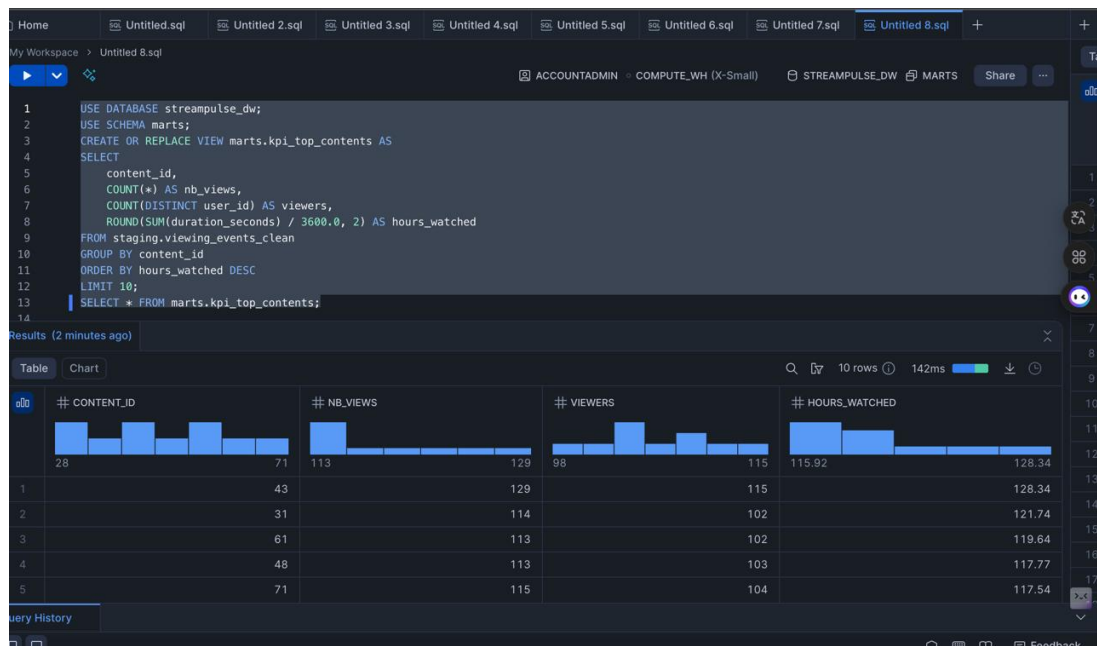
WATCH_STATUS	NB_EVENTS	PERCENTAGE
completed	2714	27.14
half	2976	29.76
short	4309	43.09



```
1 USE DATABASE streampulse_dw;
2 USE SCHEMA marts;
3 CREATE OR REPLACE VIEW marts.kpi_viewing_by_device AS
4 SELECT
5     device,
6     COUNT(*) AS total_views,
7     COUNT(DISTINCT user_id) AS unique_users,
8     ROUND(AVG(watch_ratio), 2) AS avg_completion_rate,
9     ROUND(SUM(duration_seconds) / 3600.0, 2) AS total_hours_watched
10 FROM staging.viewing_events_clean
11 GROUP BY device
12 ORDER BY total_hours_watched DESC;
13 SELECT * FROM marts.kpi_viewing_by_device;
```

Results (just now)

DEVICE	TOTAL_VIEWS	UNIQUE_USERS	AVG_COMPLETION_RATE	TOTAL_HOURS_WATCHED
smart_tv	2095	492	0.57	2082.17
web	1993	495	0.59	2015.92
mobile	2010	492	0.58	2014.96
chromecast	1941	488	0.58	1946.01
tablet	1960	487	0.57	1938.52



6 Orchestration du pipeline avec Apache Airflow

6.1 Rôle d'Apache Airflow dans l'architecture StreamPulse

Dans le projet **StreamPulse**, Apache Airflow est utilisé comme moteur d'orchestration du pipeline de données. Son rôle principal est d'automatiser et de superviser l'exécution des différentes étapes du flux de données, depuis l'extraction jusqu'à la transformation finale.

Airflow permet notamment :

- la planification automatique des traitements,
- la définition claire des dépendances entre les tâches,
- la supervision de l'exécution via une interface web,
- la gestion des erreurs et des relances.

Grâce à Airflow, le pipeline StreamPulse devient reproductible, robuste et prêt pour une exécution en environnement réel.

6.2 Choix d'un déploiement Airflow avec Docker

Afin de simplifier le déploiement et d'assurer la portabilité du projet, Apache Airflow a été déployé à l'aide de **Docker** et **Docker Compose**.

Cette approche présente plusieurs avantages :

- isolation complète de l'environnement Airflow,
- compatibilité avec plusieurs machines,
- facilité de configuration des dépendances,
- déploiement rapide sans installation locale complexe.

La structure du projet Airflow est organisée comme suit :

- `dags/` : contient les DAGs Airflow,
- `logs/` : stocke les journaux d'exécution,
- `plugins/` : extensions personnalisées (si nécessaire),
- `docker-compose.yml` : configuration des services,
- `requirements.txt` : dépendances Python.

6.3 Installation et initialisation d'Airflow

Le déploiement repose sur l'image officielle `apache/airflow:2.9.3`. L'initialisation de l'environnement est assurée par le service `airflow-init`.

Les commandes suivantes permettent d'initialiser la base Airflow et de créer l'utilisateur administrateur :

```
airflow db migrate
airflow users create \
  --username admin \
  --role Admin \
  --email admin@streampulse.com
```

L'interface web d'Airflow est ensuite accessible via le port 8080.

6.4 Configuration Docker de l'environnement Airflow

Le fichier `docker-compose.yml` définit les différents services nécessaires au bon fonctionnement d'Airflow :

- PostgreSQL : base de métadonnées Airflow,
- `airflow-init` : initialisation du système,
- `airflow-webserver` : interface utilisateur,
- `airflow-scheduler` : planification des tâches.

L'extrait suivant illustre la configuration du webserver Airflow :

```
airflow-webserver:
  image: apache/airflow:2.9.3
  depends_on:
    airflow-init:
      condition: service_completed_successfully
  environment:
    AIRFLOW__CORE__EXECUTOR: LocalExecutor
    AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:airflow@postgres/airflow
    AIRFLOW__CORE__LOAD_EXAMPLES: "false"
  volumes:
    - ./dags:/opt/airflow/dags
    - ./logs:/opt/airflow/logs
    - ./plugins:/opt/airflow/plugins
  ports:
    - "8080:8080"
  command: webserver
  restart: always
```

Cette configuration garantit une séparation claire entre l'orchestration et les autres composants du pipeline.

6.5 Gestion des dépendances Airflow

Les dépendances nécessaires au pipeline StreamPulse sont listées dans le fichier `requirements.txt` :

```
requirements.txt X
requirements.txt
1  apache-airflow-providers-postgres
2  apache-airflow-providers-amazon
3  apache-airflow-providers-snowflake
4
```

Ces providers permettent à Airflow d'interagir directement avec :

- PostgreSQL pour l'extraction,
- Amazon S3 pour le stockage intermédiaire,
- Snowflake pour le chargement et l'analyse.

6.6 Définition du DAG StreamPulse

Le cœur de l'orchestration repose sur le DAG `streampulse_dag.py`, situé dans le dossier `dags/`.

Ce DAG décrit l'ordre d'exécution logique du pipeline de données.

6.6.1 Structure générale du DAG

```
with DAG(
    dag_id="streampulse_pipeline",
    start_date=datetime(2024, 1, 1),
    schedule_interval="@daily",
    catchup=False,
    tags=["streampulse", "data-pipeline"]
) as dag:
```

Le pipeline est exécuté quotidiennement, sans reprise rétroactive des anciennes dates.

6.7 Description des tâches du pipeline

Le DAG StreamPulse est composé de quatre tâches principales.

6.7.1 Extraction depuis PostgreSQL

Cette tâche simule l'extraction des données issues de la base transactionnelle PostgreSQL.

```
def extract_from_postgres():
    print("Extraction des données depuis PostgreSQL")
```

Elle représente la récupération des tables métiers (utilisateurs, contenus, événements de visionnage).

6.7.2 Chargement vers Amazon S3

Les données extraites sont ensuite déposées dans Amazon S3, qui joue le rôle de Data Lake.

```
def load_to_s3():
    print("Chargement des données vers S3")
```

Cette étape permet de découpler la source transactionnelle du Data Warehouse analytique.

6.7.3 Chargement dans Snowflake

Une fois les données disponibles dans S3, elles sont chargées dans Snowflake via un stage externe.

```
def load_to_snowflake():
    print("Chargement des données dans Snowflake")
```

Cette tâche correspond à l'ingestion dans la couche RAW du Data Warehouse.

6.7.4 Transformation avec dbt

La dernière étape déclenche la transformation analytique à l'aide de dbt.

```
def transform_with_dbt():  
    print("Transformation des données avec dbt")
```

Les données sont nettoyées, enrichies et agrégées pour produire les Data Marts finaux.

```
streampulse_dag.py 2 X docker-compose.yml requirements.txt .env  
dags > streampulse_dag.py > ...  
1 from airflow import DAG  
2 from airflow.operators.python import PythonOperator  
3 from datetime import datetime  
4  
5 def extract_from_postgres():  
6     print("Extraction des données depuis PostgreSQL")  
7  
8 def load_to_s3():  
9     print("Chargement des données vers S3")  
10  
11 def load_to_snowflake():  
12     print("Chargement des données dans Snowflake")  
13  
14 def transform_with_dbt():  
15     print("Transformation des données avec dbt")  
16  
17 with DAG(  
18     dag_id="streampulse_pipeline",  
19     start_date=datetime(2024, 1, 1),  
20     schedule_interval="@daily",  
21     catchup=False,  
22     tags=["streampulse", "data-pipeline"]  
23 ) as dag:  
24  
25     extract = PythonOperator(  
26         task_id="extract_from_postgres",  
27         python_callable=extract_from_postgres  
28     )  
29  
30     load_s3 = PythonOperator(  
31         task_id="load_to_s3",  
32         python_callable=load_to_s3  
33     )  
34  
35     load_snowflake = PythonOperator(  
36         task_id="load_to_snowflake",  
37         python_callable=load_to_snowflake  
38     )  
39  
40     transform = PythonOperator(  
41         task_id="transform_with_dbt",  
42         python_callable=transform_with_dbt  
43     )  
44  
45     extract >> load_s3 >> load_snowflake >> transform  
46
```

6.8 Chaînage des tâches

L'ordre d'exécution du pipeline est défini explicitement dans le DAG :

```
extract >> load_s3 >> load_snowflake >> transform
```

Ce chaînage garantit :

- aucune transformation sans données chargées,
- aucune ingestion sans extraction préalable,
- une exécution séquentielle maîtrisée.

6.9 Supervision et suivi des exécutions

Grâce à l'interface web Airflow, il est possible de :

- visualiser le DAG StreamPulse,
- suivre l'état de chaque tâche,
- consulter les logs détaillés,
- relancer une tâche en cas d'échec.

Airflow joue ainsi un rôle central dans la fiabilité et l'automatisation du pipeline StreamPulse.

7 Data Mart et Visualisation des Données

7.1 Objectif de la phase Data Mart et BI

L'objectif de cette phase est de transformer les données brutes issues du Data Lake et chargées dans Snowflake en données analytiques exploitables, puis de les visualiser à l'aide d'un outil de Business Intelligence. Pour cela, nous avons conçu un **Data Mart** basé sur une architecture en étoile (*Star Schema*) et développé un tableau de bord interactif avec **Power BI**.

7.2 Construction du Data Mart dans Snowflake

7.2.1 Organisation des schémas

La base de données STREAMPULSE_DW est organisée en plusieurs schémas :

- RAW : contient les données brutes chargées depuis Amazon S3.
- MARTS : contient les tables analytiques du Data Mart.

Le schéma MARTS a été utilisé exclusivement pour la création des tables de dimension et de faits.

7.2.2 Table de faits

La table de faits FACT_VIEWING représente les événements de visionnage. Elle contient les clés de jointure vers les dimensions ainsi que les mesures quantitatives.

Principales colonnes :

- EVENT_ID : identifiant de l'événement
- CONTENT_ID : identifiant du contenu
- DEVICE : type d'appareil utilisé
- DATE_ID : date de visionnage
- DURATION_SECONDS : durée du visionnage
- WATCH_RATIO : taux de complétion du contenu

Cette table constitue le cœur analytique du Data Mart.

7.2.3 Tables de dimensions

Plusieurs tables de dimensions ont été créées afin de contextualiser les données :

Dimension Temps (DIM_DATE)

La dimension temps permet l'analyse temporelle des données. Elle est construite à partir de la date de début de visionnage.

- DATE_ID
- DAY
- MONTH
- YEAR

Dimension Appareil (DIM_DEVICE)

Cette dimension décrit le type d'appareil utilisé par l'utilisateur. Un nettoyage des données a été effectué (suppression des espaces et uniformisation en majuscules) afin de garantir l'unicité des valeurs.

Dimension Contenu (DIM_CONTENT)

La dimension contenu représente les films ou séries visionnés. Une seule ligne par CONTENT_ID a été conservée afin de respecter les règles de modélisation d'un schéma en étoile.

7.2.4 Architecture en étoile

L'ensemble du Data Mart suit une architecture en étoile, avec :

- une table de faits centrale (FACT_VIEWING),
- plusieurs tables de dimensions reliées par des relations de type *many-to-one*.

Cette structure optimise les performances analytiques et simplifie la visualisation dans Power BI.

7.3 Connexion de Power BI à Snowflake

Power BI Desktop a été connecté directement à Snowflake via le connecteur natif Snowflake. Les paramètres de connexion utilisés sont :

- Serveur Snowflake (Account Identifier)
- Warehouse : COMPUTE_WH
- Base de données : STREAMPULSE_DW
- Schéma : MARTS

Seules les tables analytiques du Data Mart ont été chargées dans Power BI.

7.4 Modélisation des données dans Power BI

Les relations suivantes ont été définies dans Power BI :

- FACT_VIEWING[DATE_ID] → DIM_DATE[DATE_ID]
- FACT_VIEWING[DEVICE] → DIM_DEVICE[DEVICE]
- FACT_VIEWING[CONTENT_ID] → DIM_CONTENT[CONTENT_ID]

Toutes les relations sont de type *Many-to-One* avec un filtrage à sens unique, respectant les bonnes pratiques BI.

7.5 Visualisation et Indicateurs Clés

Plusieurs visualisations ont été créées afin d'analyser le comportement des utilisateurs :

7.5.1 Indicateurs clés (KPIs)

- **Nombre total de visionnages** : nombre total d'événements de visionnage.
- **Taux moyen de complétion** : moyenne du WATCH_RATIO, indiquant le niveau d'engagement des utilisateurs.
- **Taux de visionnage élevé** : pourcentage de sessions avec un taux de complétion supérieur à 70%.

7.5.2 Analyses graphiques

- Répartition des visionnages par appareil.
- Taux moyen de complétion par type d'appareil.
- Top 5 des contenus les plus visionnés.
- Évolution du nombre de visionnages dans le temps.
- Distribution de la durée des sessions de visionnage.

Ces visualisations permettent de dégager des insights métier sur les usages, l'engagement et les préférences des utilisateurs.

